
SOFTWARE REQUIREMENTS SPECIFICATION

FOR

UniMIS - Faculty Management Information System

DEPARTMENT OF INFORMATION AND

COMMUNICATION TECHNOLOGY

FACULTY OF TECHNOLOGY - UNIVERSITY OF RUHUNA

LEVEL II (Semester I)

ICT2132- Object Oriented Programming Practicum

Mini Project - Submission 01

Prepared By:

TG No	Name
TG/2024/2067	Dasindu Dilvan
TG/2024/2106	Praveen Sandeepa
TG/2024/2107	Dulsha Hemini
TG/2024/2120	T.V.K. Sendiya

Table of Contents

1. INTRODUCTION	3
1.1 Purpose	3
1.2 Scope	3
1.3 Definitions, Acronyms and Abbreviations	3
1.4 References	3
2. OVERALL DESCRIPTION.....	4
2.1 Product Perspective	4
2.2 Product Functions.....	4
2.3 User Classes and Characteristics.....	4
2.4 Operating Environment	4
2.5 Assumptions and Dependencies	5
3. SPECIFIC REQUIREMENTS.....	5
3.1 Functional Requirements	5
3.1.1 Authentication Requirements	5
3.1.2 Administrator Requirements	5
3.1.3 Lecturer Requirements	5
3.1.4 Technical Officer Requirements	5
3.1.5 Undergraduate Requirements	6
3.1.6 Course Management Requirements.....	6
3.1.7 Attendance Management Requirements.....	6
3.1.8 Academic Information Requirements	6
3.2 Non-Functional Requirements.....	7
3.2.1 Performance and Reliability.....	7
3.2.2 Security.....	7
3.2.3 Usability and Maintainability	7
3.3 OOP Concepts Mapping.....	7
3.4 Database Requirements	8
4. USE CASES AND SYSTEM SCENARIOS.....	8
4.1 System Actors	8
4.2 Major Use Cases	8
4.3 Use Case Descriptions.....	9
5. APPENDICES	11
5.1 Glossary	11
5.2 Supporting Information	11
6. Conclusion.....	11

1. INTRODUCTION

1.1 Purpose

This SRS defines the complete requirements for UniMIS, a system built for the Faculty of Technology to manage user profiles, course details, undergraduate marks, attendance, notices, timetables, and medical records. It is written for the development team, course instructors, and evaluators, following IEEE 830-1998 and IEEE 29148-2018.

1.2 Scope

UniMIS is a **desktop-based application** (Java, Swing+FlatLef GUI, MySQL database) serving four user types: Admin, Lecturer, Technical Officer, Undergraduate. It covers:

- User profile management for all four roles
- Course creation/management with evaluation criteria
- Marks recording, grade calculation, GPA computation
- Attendance tracking (theory + practical)
- Medical record submission/approval
- Notice creation and distribution
- Timetable creation and viewing
- Eligibility checking (attendance + CA marks) for final exams

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
UniMIS	University Management Information System
SRS	Software Requirements Specification
UGC	University Grants Commission (Sri Lanka)
CA	Continuous Assessment
GPA / SGPA / CGPA	Grade Point Average / Semester GPA / Cumulative GPA
GPV	Grade Point Value
CRUD	Create, Read, Update, Delete
JDBC	Java Database Connectivity
DAO	Data Access Object
NIC	National Identity Card
PK / FK	Primary Key / Foreign Key

1.4 References

- IEEE Std 830-1998 - Recommended Practice for SRS
- UGC Commission Circular No. 12-2024 - Grading System for Sri Lankan Universities
- ICT2132 Mini Project Guidelines (2026)
- Faculty of Technology Current Semester Academic Timetable

2. OVERALL DESCRIPTION

2.1 Product Perspective

UniMIS is a new, self-contained desktop application built for the ICT2132 mini project - not a replacement for an existing system. Architecture: **two-tier** (Java client ↔ MySQL via JDBC), with the client internally layered into presentation (GUI), business logic, and data access (DAO).

2.2 Product Functions

No	Feature	Description
F1	User Profile Management	Role-based profiles for all four user types
F2	Course Management	Courses with credits, components, evaluation criteria, lecturer assignment
F3	Marks Management	CA + final marks, weighted totals
F4	Grade & GPA Calculation	Auto-grading per UGC Circular 12-2024; SGPA/CGPA
F5	Attendance Management	Theory/practical session tracking, percentages, summaries
F6	Medical Records	Submission, approval status, linked to missed attendance
F7	Eligibility Checking	Attendance $\geq 80\%$ AND CA $\geq 40\%$
F8	Notice Management	Categorized, targeted notices
F9	Timetable Management	Weekly timetables with conflict validation
F10	Reporting	Individual/batch reports across marks, grades, attendance, eligibility

2.3 User Classes and Characteristics

Role	Count	Key Responsibilities
Admin	1	Full access - manages all profiles, courses, notices, timetables
Lecturer	≥ 5	Course materials, mark upload, views student/eligibility/grade/GPA/attendance/medical data, notices
Technical Officer	≥ 4	Attendance & medical records, views notices and department timetables
Undergraduate	≥ 20	Updates only contact info/photo; views own attendance, medicals, courses, grades, GPA, timetable, notices

2.4 Operating Environment

Component	Requirement
OS	Windows, macOS, Linux
Language	Java
GUI	Swing + FlatLaf
Database	MySQL
Connectivity	JDBC
Version Control	Git + GitHub

2.5 Assumptions and Dependencies

Assumptions: single pre-created Admin account; current semester timetable used as course reference; theory and practical each have exactly 15 sessions; each course has coordinator-defined evaluation criteria; users have basic computer literacy; MySQL server reachable on localhost/network.

Dependencies: JRE 8+, MySQL Server, MySQL JDBC Connector on class path, UGC Circular No. 12-2024.

3. SPECIFIC REQUIREMENTS

3.1 Functional Requirements

3.1.1 Authentication Requirements

- All users (Admin, Lecturer, Technical Officer, Undergraduate) shall log in using username and password
- All users shall log out securely and return to the login screen
- Each user shall only access features authorized for their role

3.1.2 Administrator Requirements

- View a dashboard with system summary statistics
- Update their own profile details and password
- Create, view, update, and delete Lecturer profiles
- Create, view, update, and delete Technical Officer profiles
- Create, view, update, and delete Undergraduate profiles
- Search and filter all user profiles by role, department, batch, or status
- Deactivate or delete user accounts with dependency warnings
- Create, view, update, and delete notices with categories and target audiences
- Create, view, update, and delete timetable entries with conflict validation

3.1.3 Lecturer Requirements

- Update their own contact details, designation, specialization, and profile picture
- Add or modify course materials for assigned courses
- View details of students enrolled in their courses
- View notices targeted to Lecturers or All users

3.1.4 Technical Officer Requirements

- Update their own contact details, assigned labs, and profile picture
- View notices targeted to Technical Officers or All users
- View the timetable of their assigned department

3.1.5 Undergraduate Requirements

- Update only their phone, email, address, and profile picture
- View details of their enrolled courses
- View their own weekly timetable
- View notices targeted to Undergraduates or All users

3.1.6 Course Management Requirements

- Admin shall create courses with code, name, credits, and session details
- Admin shall define CA and Final Exam evaluation criteria (weightages must total 100%)
- Users shall view courses with search and filter options
- Admin shall update existing course details and recalculate hours
- Admin shall delete courses only if no dependent records exist
- Admin shall assign one or more lecturers to a course (Theory/Practical/Both)
- Admin shall enroll students into courses individually or by batch

3.1.7 Attendance Management Requirements

- Technical Officer shall mark student attendance as Present, Absent, or Medical per session
- Technical Officer shall edit previously recorded attendance entries
- Technical Officer and Lecturer shall view batch and individual attendance (Theory, Practical, Combined)
- Undergraduate shall view their own attendance percentages per course

3.1.8 Academic Information Requirements

- Lecturer shall upload CA and Final Exam marks (out of 100) for assigned courses
- Lecturer shall view CA marks summary for individuals and batch
- Lecturer shall view weighted final marks of students
- Lecturer and Undergraduate shall view course grades based on UGC Circular No. 12-2024
- Undergraduate shall view their SGPA and CGPA; Lecturer shall view for their students
- System shall check and display exam eligibility (Attendance $\geq 80\%$ and CA $\geq 40\%$)
- Technical Officer shall add, update, and approve/reject medical records
- Lecturer and Undergraduate shall view relevant medical records and approval status

3.2 Non-Functional Requirements

3.2.1 Performance and Reliability

- All database operations (Add, View, Update, Delete) shall complete within 3 seconds
- System shall handle 30+ user records and full semester data without lag
- System shall validate all inputs before saving and shall never crash on invalid data
- Database shall enforce Primary Keys, Foreign Keys, and Unique Constraints
- The system shall use try-catch-finally blocks and custom exceptions (DuplicateEntryException, InvalidInputException, TimetableConflictException, etc.) to handle errors safely without crashing, displaying user-friendly popup messages.

3.2.2 Security

- All passwords shall be stored in hashed, never in plain text
- All database queries shall use Prepared Statements to prevent SQL injection
- Each user shall only access features allowed for their role

3.2.3 Usability and Maintainability

- GUI shall be simple, consistent, and easy to use for first-time users
- All error messages shall be clear and user-friendly (no raw stack traces)
- Code shall follow OOP principles and layered architecture (Models, Views, Controllers, DAOs)
- Application shall run on Windows, macOS, or Linux with JDK 8+ and MySQL 5.7+ as a standalone desktop app

3.3 OOP Concepts Mapping

- Classes & Objects: Create separate Java classes for each entity such as Admin, Lecturer, TechnicalOfficer, Undergraduate, Course, Attendance, and Marks.
- Inheritance: Create a parent class User with common fields (name, NIC, email) and extend it to create Admin, Lecturer, Technical Officer, and Undergraduate subclasses.
- Abstraction: Use an abstract class User with abstract methods like getRole() and an interface CRUD Operations for database actions.
- Polymorphism: Override the getRole() method in each subclass to return specific role names; overload search methods to accept different parameters (String, int).
- Encapsulation: Declare all class attributes as private and access them only through public getters and setters with input validation.
- Exception Handling: Use try-catch-finally blocks for all database operations and create custom exceptions like Duplicate Entry Exception and Invalid Input Exception.
- Database Handling: Use a singleton Database Connection class and the DAO (Data Access Object) pattern with Prepared Statement for all queries.
- GUI Implementation: Build the interface using Java Swing components like JFrame, JPanel, JTable, JButton, and JOptionPane for dialogs.

3.4 Database Requirements

UniMIS utilizes a relational database management system (**MySQL Server 5.7+**) to ensure secure, consistent, and structured data storage. The database design adheres to relational database principles with standard normalization rules.

Key Database Domains

- **User Management:** Stores personal profiles, authentication credentials, and role-specific information for Admins, Lecturers, Technical Officers, and Undergraduates.
- **Course & Curriculum Management:** Manages course details, credit allocations, lecture assignments, student enrollments, and continuous assessment (CA) evaluation criteria.
- **Attendance & Medical Tracking:** Records per-session attendance (Theory and Practical) and tracks student medical certificates with approval workflows.
- **Academic Performance:** Stores CA and final exam marks, calculated grades according to UGC Circular No. 12-2024, and semester/cumulative GPA records.
- **System Scheduling & Communication:** Handles weekly department timetables, lecture venues, and published academic notices.

Data Integrity & Connectivity

- **Referential Integrity:** Enforced using Primary Keys (PK), Foreign Keys (FK), and Unique constraints across all entities to prevent data duplication.
- **Database Connectivity:** The Java application interacts with the MySQL database via JDBC (Java Database Connectivity) using the DAO (Data Access Object) pattern and parameterized queries for secure data access.

4. USE CASES AND SYSTEM SCENARIOS

4.1 System Actors

Admin (1), Lecturer (≥ 5), Technical Officer (≥ 4), Undergraduate (≥ 20).

4.2 Major Use Cases

Actor	Main Use Case Groups
Admin	Authentication, Dashboard, User Management, Course Management, Notice Management, Timetable Management, Profile Management
Lecturer	Authentication, Profile Management, Course Materials, Marks Management, Student Information, Attendance & Medicals, Notices
Technical Officer	Authentication, Profile Management, Attendance Management, Medical Management, Notices, Timetable
Undergraduate	Authentication, Profile Management, Attendance, Medicals, Course Information, Grades & GPA, Timetable, Notices

4.3 Use Case Descriptions

Administrator

- **Login** - Admin enters username and password → System validates credentials → Dashboard opens. If invalid, error message is shown.
- **Dashboard** - Admin views summary statistics including total users, courses, and active notices.
- **Manage Users** - Admin creates, views, updates, or deletes Lecturer, Technical Officer, and Undergraduate profiles. Duplicate NIC, username, or email is blocked.
- **Manage Courses** - Admin enters course code, name, credits → Sets exam weightages (must total 100%) → Assigns lecturers → Enrolls students → Saves. If weightage $\neq$ 100%, save is blocked.
- **Manage Notices** - Admin creates notices with title, content, category, and target audience → Sets expiry date → Publishes. Can also update or delete existing notices.
- **Manage Timetables** - Admin selects batch, course, day, time, venue, and lecturer → System checks for time slot conflicts → Saves valid entry. Overlapping slots are blocked.
- **Update Profile** - Admin updates own contact details and password.
- **Logout** - Admin clicks logout → Session ends → Login screen appears.

Lecturer

- **Login** - Lecturer enters credentials → System authenticates → Dashboard opens. If invalid, error message is shown.
- **Update Profile** - Lecturer updates contact info, designation, specialization, and profile picture. Username and password cannot be changed.
- **Course Materials** - Lecturer selects assigned course → Uploads or updates notes, references, and links → Saves.
- **Upload Marks** - Lecturer selects course and exam component → Enters marks (0-100) per student → System validates and saves → Grades and GPA auto-update. Marks outside 0-100 are rejected.
- **View Students** - Lecturer selects course/batch → System displays student list with CA marks, final grades, GPA, and exam eligibility.
- **Attendance & Medicals** - Lecturer selects course → System displays individual and batch attendance percentages and medical records.
- **Notices** - Lecturer views active notices targeted to Lecturers or All users.
- **Logout** - Lecturer clicks logout → Session ends → Login screen appears.

Technical Officer

- **Login** - TO enters credentials → System authenticates → Dashboard opens. If invalid, error message is shown.
- **Update Profile** - TO updates contact info, assigned labs, and profile picture. Username and password cannot be changed.
- **Manage Attendance** - TO selects course, session type, and date → Marks each student as Present, Absent, or Medical → System saves and recalculates attendance percentages.
- **Manage Medicals** - TO enters student Reg. No., date range, reason → Sets approval status (Pending/Approved/Rejected) → Saves. Invalid Reg. No. is rejected.
- **Notices** - TO views active notices targeted to Technical Officers or All users.
- **Timetable** - TO selects department and semester → System displays weekly lab and session timetable.
- **Logout** - TO clicks logout → Session ends → Login screen appears.

Undergraduate

- **Login** - Student enters credentials → System authenticates → Dashboard opens. If invalid, error message is shown.
- **Update Contacts** - Student updates only phone, email, address, and profile picture. Name, username, password, and registration number are locked.
- **Attendance** - Student selects course → System displays Theory, Practical, and Combined attendance percentages.
- **Medicals** - Student views submitted medical records with approval status (Pending/Approved/Rejected).
- **Courses** - Student views enrolled course details including code, credits, lecturers, and evaluation criteria.
- **Grades & GPA** - Student views marks, course grades, SGPA, and CGPA based on UGC grading scale. If marks are not uploaded yet, "No results available" is shown.
- **Timetable** - Student views weekly timetable matching their batch and semester.
- **Notices** - Student views active notices targeted to Undergraduates or All users.
- **Logout** - Student clicks logout → Session ends → Login screen appears.

5. APPENDICES

5.1 Glossary

See §1.3 for core acronyms. Additional terms:

- **Grade Point Value (GPV):** numeric weight assigned to each letter grade.
- **Batch Missed student:** an undergraduate whose original batch/semester progression was interrupted, tracked via enrollment_status.
- **DAO (Data Access Object):** design pattern isolating database access per entity.

5.2 Supporting Information

- UGC Circular No. 12-2024 Grading Scale

GPA Formulas

- $SGPA = \Sigma(\text{Credit} \times GPV) \div \Sigma(\text{Credits})$ for that semester.
- $CGPA = \Sigma(\text{Credit} \times GPV) \div \Sigma(\text{Credits})$ across all semesters.
- $\text{Final Mark} = \Sigma(\text{component mark} \times \text{weightage}\% \div 100)$
- Example: Quiz(10%)=80, Assignment(15%)=70, Mid-term(15%)=60, Final(60%)=75
→ $\text{Final Mark} = 8 + 10.5 + 9 + 45 = 72.5 \rightarrow \text{Grade A-}$.

Eligibility Scenarios Covered by Test Data: >80% attendance; exactly 80%; <80% without medical; >80% with medical; <80% with medical.

6. Conclusion

This SRS describes the main requirements of **UniMIS**, a University Management Information System for the Faculty of Technology, University of Ruhuna.

It defines the functions available to Admins, Lecturers, Technical Officers, and Undergraduates. The system manages users, courses, marks, grades, GPA, attendance, medical records, notices, and timetables.

The document also includes the database, security, error handling, and system quality requirements. UniMIS will be developed using Java, MySQL, and OOP concepts while following relevant IEEE guidelines and the UGC grading system. This SRS will guide the development and evaluation of the ICT2132 OOP Practicum Mini Project.